CHAPTER 1

INTRODUCTION

1.1 Motivation

Currently, most people have to book event halls using old, manual methods like making phone calls or visiting the venue in person many times. This creates many problems because everything is done on paper, which is slow and makes it easy to lose important information. Often, customers cannot check if a hall is free for their date without asking the manager, and they cannot see photos or prices clearly. This manual way of working often leads to mistakes like "double-booking" the same hall for two people at once, which causes a lot of confusion and stress for both the hall owner and the customer.

The main reason for building this project is to make hall booking fast, simple, and digital. Instead of wasting time on paperwork, this system allows users to see all available halls, look at high-quality photos, and check prices directly on their phone or computer. Features like easy login with Email **or Phone OTP** and simple payment options for **UPI and Cards** make the process very reliable. By using this system, a user can book a perfect hall for their event in just a few minutes, while the hall manager can easily keep track of all bookings and payments in one place without any confusion.

1.2 Project Scope

This project is basically updating the manual hall booking process into an internet-based application so that users can check hall photos, real-time availability, and their booking details online. It is a multi-user version and can take care of all the fundamental functions of venue management like Hall Cataloging (listing halls), Slot Booking, Payment Tracking, and Membership Management. It can successfully handle all the basic needs of event organizers and hall managers in a single, organized platform.

1.3 Project Stakeholders

The project stakeholders for this document are:

- Software Project Managers
- Software Engineers
- Software Developers
- Venue Managers and Administrators
- Event Organizers and Customers

1.4 Project Scheduling

Phase	Tasks	Duration
Week 1-2	Requirement Analysis & Feasibility Study	2 Weeks
Week 3	System Design (ERD, DFD, UML Diagrams)	1 Week
Week 4-6	Database Design & Setup (MySQL)	3 Weeks
Week 7-10	Frontend Development (React & Tailwind CSS)	4 Weeks
Week 11-14	Backend Development (Node.js & Express API)	4 Weeks
Week 15-16	Integration & Testing	2 Weeks
Week 17	Deployment & Documentation	1 Week
Week 18	Final Review & Submission	1 Week

CHAPTER 2

SYSTEM ANALYSIS

2.1 Introduction

The Hall Booking System is an application designed to replace manual venue booking methods with a fast, internet-based platform. This newly created system is perfect for managing private halls, banquet centers, and event spaces. It allows you to store detailed information about the halls and their amenities while easily controlling the booking schedules and payment tracking in one place.

2.2 Requirement Analysis

2.2.1 Functional Requirement

The Administrator (Hall Manager)

- **Add Hall**: New halls and venues must be entered in the database.
- **Update Hall**: Any changes in hall prices, photos, or features should be updated.
- **Delete Hall/Entry**: Incorrect or old hall entries must be removed from the system.
- **Inquiry Users**: View details of all registered members to see their profiles.
- **Inquiry Bookings**: View and check all current and past booking records in the database.
- **Confirm Booking**: To finalize a request, the booking must be confirmed by the admin.
- **Complete Booking**: After the event is over, the system will mark the booking as completed.
- **Inquiry Pending Requests**: Admin generates a list of all new booking or membership applications waiting for approval.
- **Block Room/Hall**: Admin can manually reserve or block a hall for maintenance or special repairs.
- **Manage Permissions**: Admin can change user roles or enable/disable a user's account.

Member

- **Authentication:** User must log in (via Google, OTP, or Password) before using the system.
- **Search Hall:** User can search for halls by location, price, and capacity.
- **Book Hall:** After finding a suitable hall, the member can mark it as a requested booking.
- **Check Dashboard:** Member can check their booking status, payment history, and profile details.
- **Manage Favourite's: Users can add halls to their "Saved" list to book them easily in the future.**
- **Handle Payments: Add or update payment methods like UPI IDs or Card details for quick checkout**

The Staff (Assistant Manager)

- **Manage Database:** Update venue information and maintain the hall listings.
- **Monitor Logs:** Review user activity logs to ensure the system is being used safely and correctly.
- **Support System:** Help in answering user inquiries and managing the "Contact Us" notifications.

Guest (New Visitor)

- **Register User**: New users must fill out the registration form to create an account.
- **Browse Venues:** Guests can view the public landing page to browse hall galleries without logging in.
- **Check Prices**: View the hourly rates and basic amenities of different halls listed on the platform.
- **View Availability:** Check which dates are available or booked on the public calendar.
- **Contact Support:** Use the contact form to ask questions or request more information from the admin.

2.2.2 Non-Functional Requirement

Performance Requirements

- The system should handle at least 100 concurrent users at the same time without any performance degradation.
- Hall search and filtering functionality should return accurate results within 2 seconds.
- The response time for creating a booking or updating a payment status should be less than 3 seconds.

Scalability

- The system should support the addition of many new users, halls, and thousands of transactions without requiring major code changes.
- The database should be capable of storing at least 50,000 hall listings and booking records comfortably.
- The system architecture should allow for adding new modules (like "Catering" or "Event Styling") in the future.

Security

- User authentication should be implemented using Role-Based Access Control (RBAC) for Admin and User roles.
- All user passwords must be stored using strong encryption and hashing techniques (e.g., bcrypt).
- Only authorized users (Admins) have the permission to add, edit, or remove hall data.
- All critical transactions and status changes must be logged in the Activity Logs for auditing and security purposes.

Reliability & Availability

- The system should be available 24/7 to allow users to book halls at any time.
- The system should be reliable, with a basic backup strategy to ensure no booking data is lost in case of a server issue.
- All interactive elements must demonstrate high stability and provide clear feedback to the user on every action.

User Interface & Usability

- The design should use modern Glass morphism aesthetics to provide a premium and intuitive experience.
- The website must be fully Responsive, working perfectly on mobile phones, tablets, and laptops.
- Navigation should be simple so that even non-technical users can book a hall without difficulty.

2.2.3 Software & Hardware Requirements

User	Particulars	Client System	Server System
User	Particulars	Client System (User/Manager)	Server System (Cloud/Live)
Admin & Managers (Web Dashboard)	Operating System	Windows 10/11 or macOS	Ubuntu 22.04 LTS (Linux Server)
	Processor	i3 / i5 2.0 GHz or higher	Quad-Core Xeon / vCPU
	Hard Disk	50 GB Minimum	20 GB SSD (For Database & Code)
	RAM	4 GB (Minimum)	8 GB (Recommended)
	Browser	Google Chrome (Latest Version)	Node.js v18 or modern
Members (Web App - iOS & Android)	Operating System	iOS 14+, Android 9+, Windows	Cloud Hosting (e.g., Render/Heroku)
	Device	iPhone, iPad, or Android Phone	MongoDB / MySQL (Managed)
	Connectivity	Stable 4G / 5G / WiFi	HTTPS / SSL Security
	Browser	Safari (on iOS), Chrome, Edge	API Gateway
	RAM	2 GB or higher	

2.3 Design and Implementation Constraints

The **Administrator** and **Staff** members must keep their login credentials and passwords strictly confidential to protect sensitive booking data. Only the **Admin** has the authority to manage core resources like adding new halls, updating prices, or modifying venue galleries. To ensure data privacy, every user must have a unique **Individual ID** (via email or phone) to access their private account information and booking history. Furthermore, only the **Administrator** has the permission to approve new registration requests or issue premium memberships (Silver, Gold, Platinum) to guest users.

2.4 Assumptions and Dependencies

- **Internet Connectivity**: A stable internet connection is required for both users and administrators to communicate with the central hall booking server in real-time.
- **Web Dashboard**: The Administrator and Staff must use the web-based management dashboard to update hall details, manage resources, and approve booking requests.
- **Digital Authentication**: The system is built as a digital-first platform using Secure OTP and Google Auth; therefore, users are assumed to have a valid phone number or email for login.
- **Unique User IDs**: Each registered member must have a unique User ID to access their personalized dashboard, manage their favourites, and track their booking history.
- **System Compatibility**: It is assumed that users will access the system through modern web browsers such as Google Chrome, Safari, or Microsoft Edge for the best experience.

CHAPTER 3

SYSTEM DESIGN

3.1 UML Diagram

3.1.1 Data-Flow Diagram

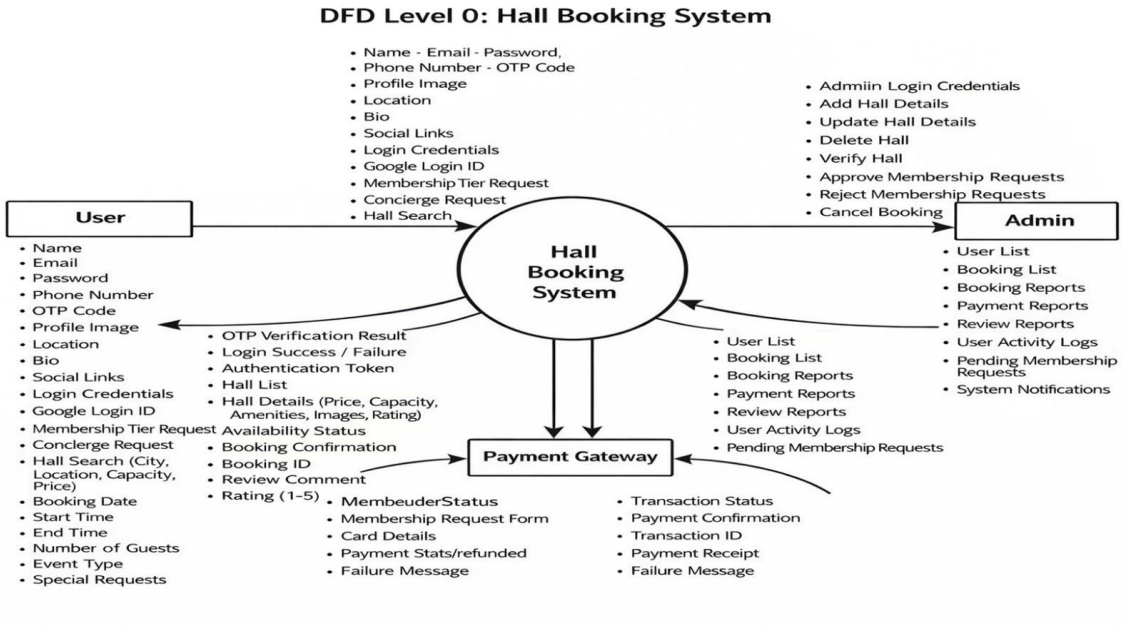


Figure 3.1.1.1 dfd level-0 diagram of hall booking system

3.1.2 Data-Flow Diagram

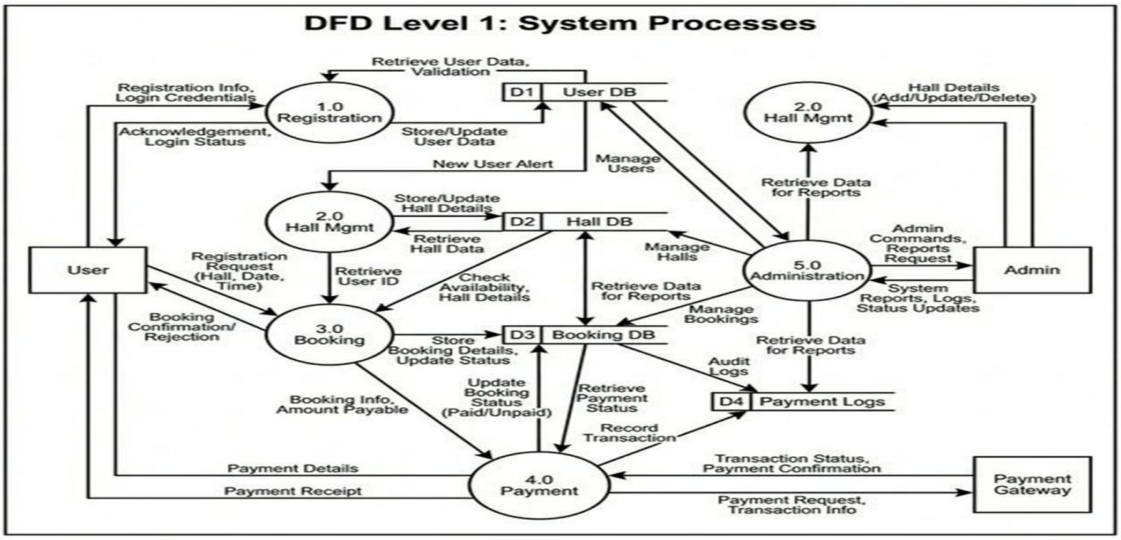


Figure 3.1.1.2 1 dfd level-1 diagram of hall booking system

3.1.3 Use-case Diagram

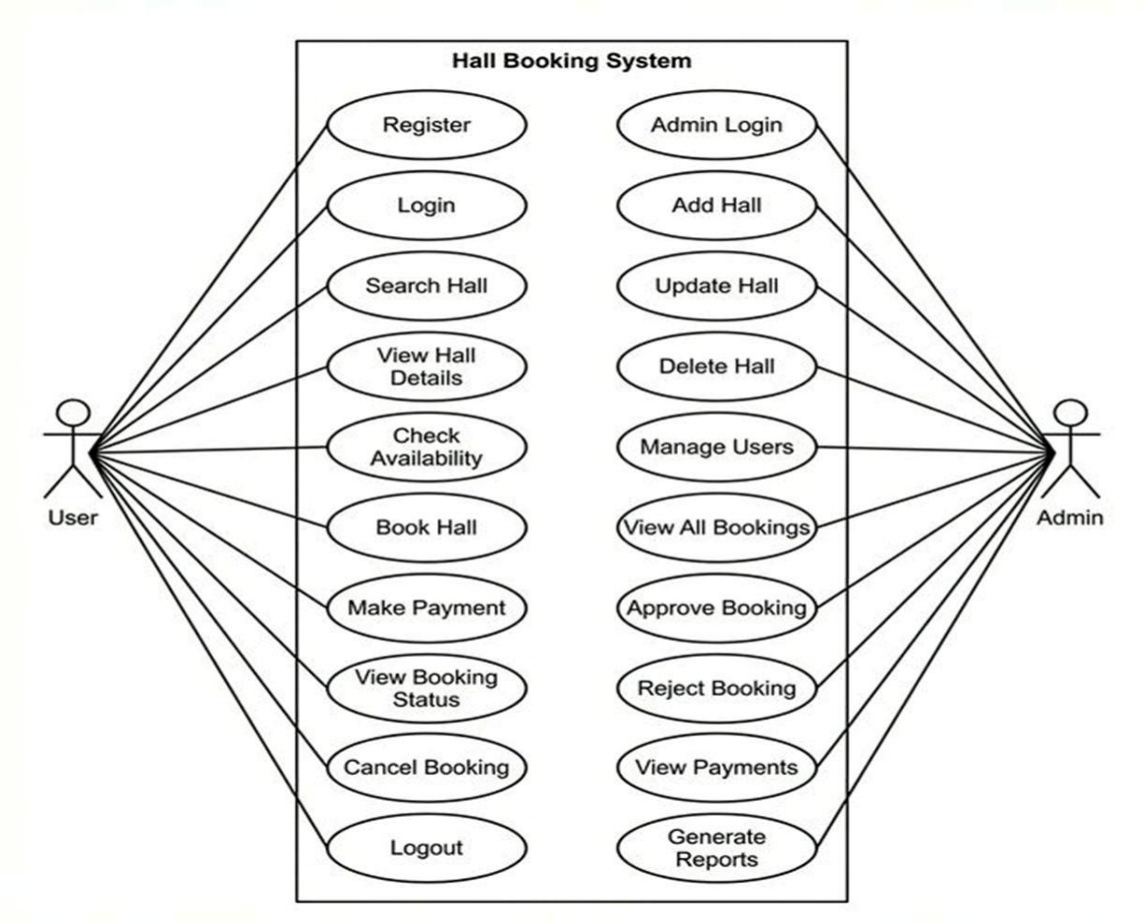


Figure 3.1.2.1 Use case diagram of hall booking system

3.1.4 Use-case Scenarios

Register

Abbreviated Title	Register
Actors	Guest
Description	Allows a new visitor to create a secure account to access booking features.
Pre-Conditions	User must have a valid Email or Phone number.
Task Sequence	• System shows Registration Form • User enters Name, Email, and Password • System saves data and creates a unique User ID
Post Conditions	User account is created, and they are redirected to the Login page.
Exception	If the Email already exists, the system displays "Account already exists."

Login

Abbreviated Title	Login
Actors	User, Admin
Description	Validates the user's identity to grant access to private dashboards.
Pre-Conditions	User must have a registered account.
Task Sequence	• System shows Login Screen • User enters Email and Password • System verifies credentials against the database
Post Conditions	User/Admin is logged in and redirected to their specific dashboard.
Exception	If passwords mismatch, system shows "Invalid Credentials" and resets form.

Search Hall

Title	Search Hall
Actors	User, Guest
Description	Allows searching for venues using filters like City, Price, and Capacity.
Pre-Conditions	User must be on the Explore/Halls page.
Task Sequence	• System shows filters • User selects Location or Budget • System displays matching Hall results
Post Conditions	User views a list of venues that meet their needs.
Exception	If no results found, system suggests broadening the filters.

View Hall Details

Abbreviated Title	View Hall Details
Actors	User, Guest
Description	Displays comprehensive info, gallery images, and amenities for a specific hall.
Pre-Conditions	User selects a hall from the search results.
Task Sequence	• User clicks on a Hall card • System fetches Hall info, Images, and Price details • Displays the Hall Details Page
Post Conditions	User can proceed to check availability or book.
Exception	-

Check Availability

Abbreviated Title	Check Availability
Actors	User
Description	Real-time check to see if a hall is free on a specific date/time slot.
Pre-Conditions	User must be on the specific Hall's page.
Task Sequence	• • User selects a Date and Time Slot • System queries the 'Bookings' table for conflicts • Displays "Available" or "Booked" status
Post Conditions	User can proceed to the booking form if available.
Exception	If the date is a past date, the system blocks the request.

Book Hall

Abbreviated Title	Book Hall
Actors	User
Description	Creates a reservation request for a selected venue and time.
Pre-Conditions	User must be logged in and the slot must be available.
Task Sequence	• User fills the booking form (Guest count, event type) • User submits the request • System creates a 'Pending' record
Post Conditions	Admin receives a notification; User sees booking in their dashboard.
Exception	If user is not logged in, system redirects to Login page.

Make Payment

Abbreviated Title	Make Payment
Actors	User
Description	Enables secure payment of booking fees via UPI, Card, or Net Banking.
Pre-Conditions	Booking must be confirmed or ready for payment.
Task Sequence	• User selects a Payment Method • User completes the transaction • System verifies payment and updates status to 'Paid'
Post Conditions	An automated invoice is generated for the user.
Exception	If payment fails, system displays "Transaction Failed" and allows retry.

View Booking Status

Abbreviated Title	View Status
Actors	User
Description	Allows users to track their pending, confirmed, or rejected bookings.
Pre-Conditions	User must be logged in.
Task Sequence	• User navigates to 'My Bookings' • System retrieves all records for that User ID • Displays live status (Pending/Confirmed/Cancelled)
Post Conditions	User stays updated on their event schedule.
Exception	-

Add / Update Hall

Abbreviated Title	Manage Inventory
Actors	Admin
Description	Admin adds new venues or updates existing hall prices and photos.
Pre-Conditions	Admin must be logged in to the Admin Panel.
Task Sequence	• Admin enters Hall details and uploads images • Admin clicks Save • System updates the public hall database
Post Conditions	New data is instantly live on the website.
Exception	-

Approve / Reject Booking

Abbreviated Title	Moderate Bookings
Actors	Admin
Description	Admin manages the incoming requests from users to finalize hall slots.
Pre-Conditions	There must be 'Pending' bookings in the system.
Task Sequence	• Admin reviews booking details • Admin clicks 'Approve' or 'Reject' • System updates status in the database
Post Conditions	User is notified of the decision; status changes on their dashboard.
Exception	-

Manage Users (Permissions)

Abbreviated Title	Manage Users
Actors	Admin
Description	Admin can view user profiles, change roles, or block/enable accounts.
Pre-Conditions	Admin must be logged in.
Task Sequence	• Admin selects a user from the member list • Admin updates permissions (e.g., granting Staff role) • System saves changes
Post Conditions	User permissions are updated instantly.
Exception	-

Generate Reports

Abbreviated Title	Generate Reports
Actors	Admin
Description	Generates summaries of total revenue, bookings, and popular halls.
Pre-Conditions	Data must exist in the database.
Task Sequence	• Admin selects a date range • System calculates revenue and counts bookings • Displays a summary report for management
Post Conditions	Admin views the analytical growth of the platform.
Exception	-

3.1.5 Activity Diagram

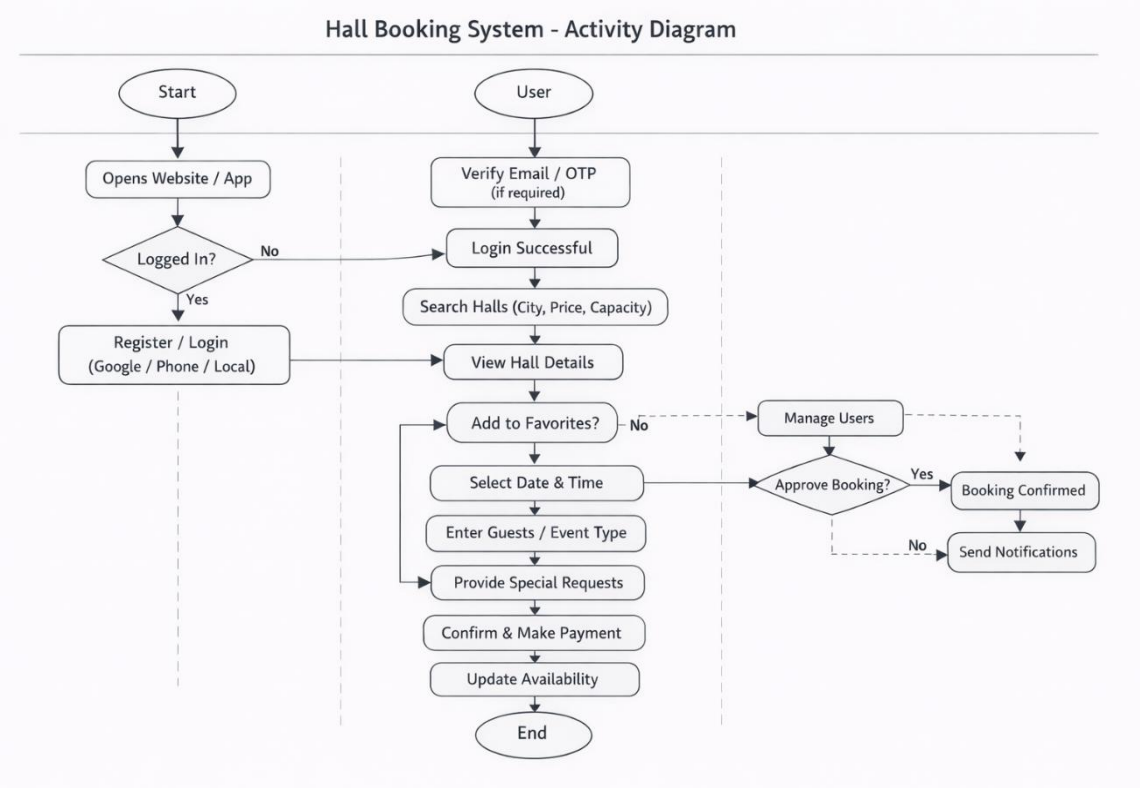


Figure 3.1.4.1 Activity diagram for hall booking

3.1.6 Sequence Diagram

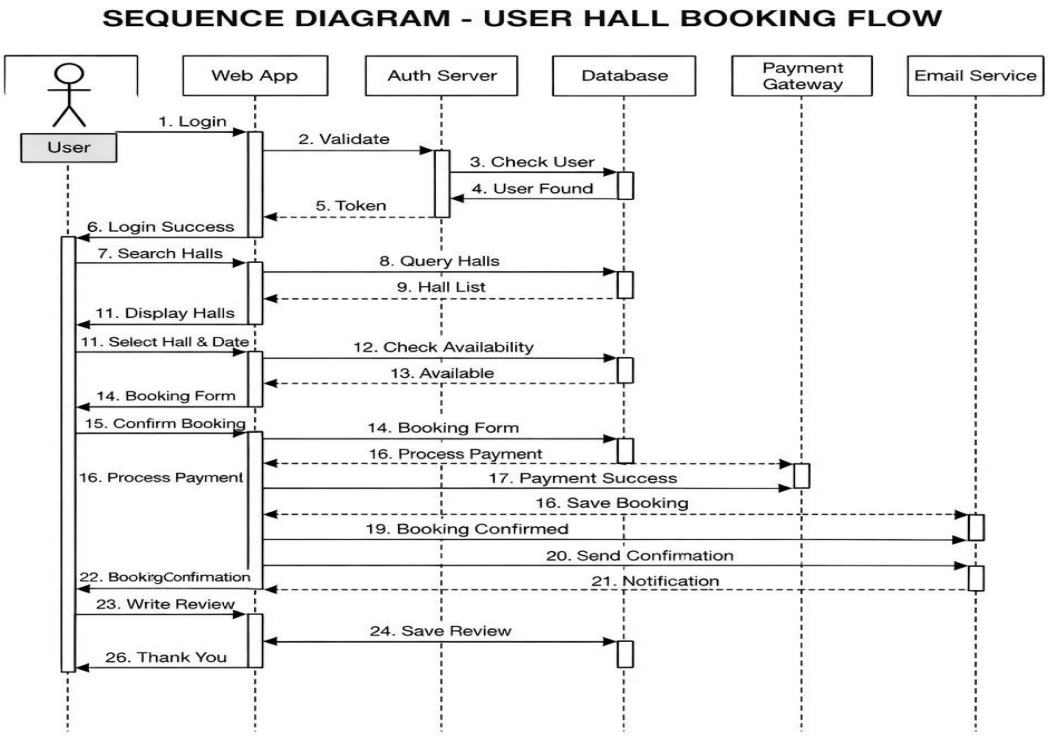


Figure 3.1.5.1 Sequence diagram for hall booking system

3.1.7 Sequence Diagram

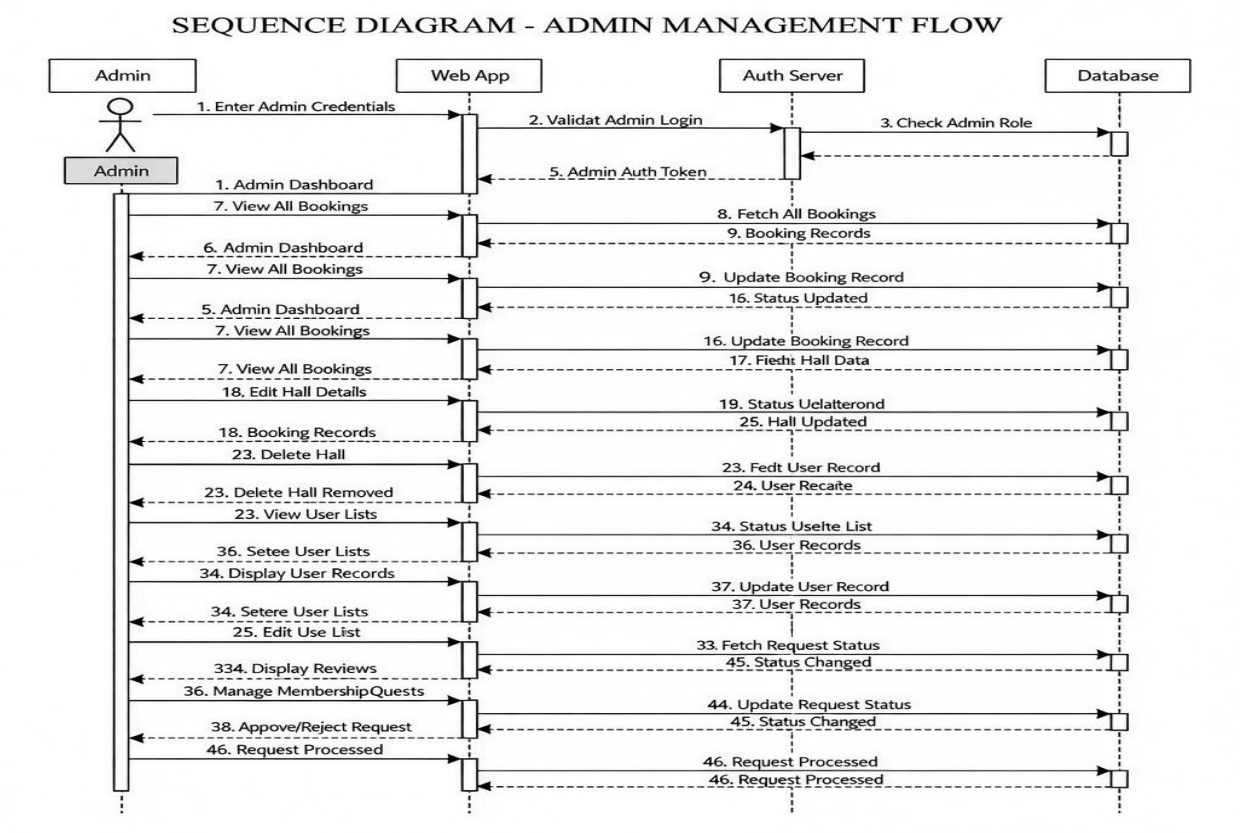


Figure 3.1.5.2 Sequence diagram for hall booking system

3.1.8 State Diagram

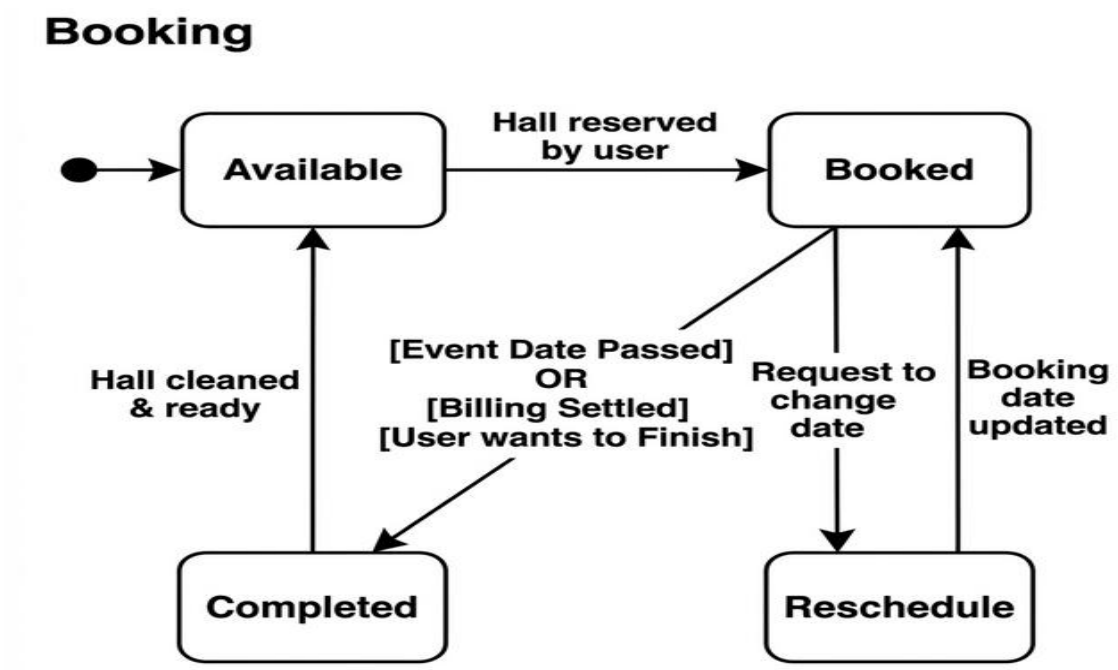


Figure 3.1.6.1 State diagram of hall booking system

3.1.9 Class Diagram

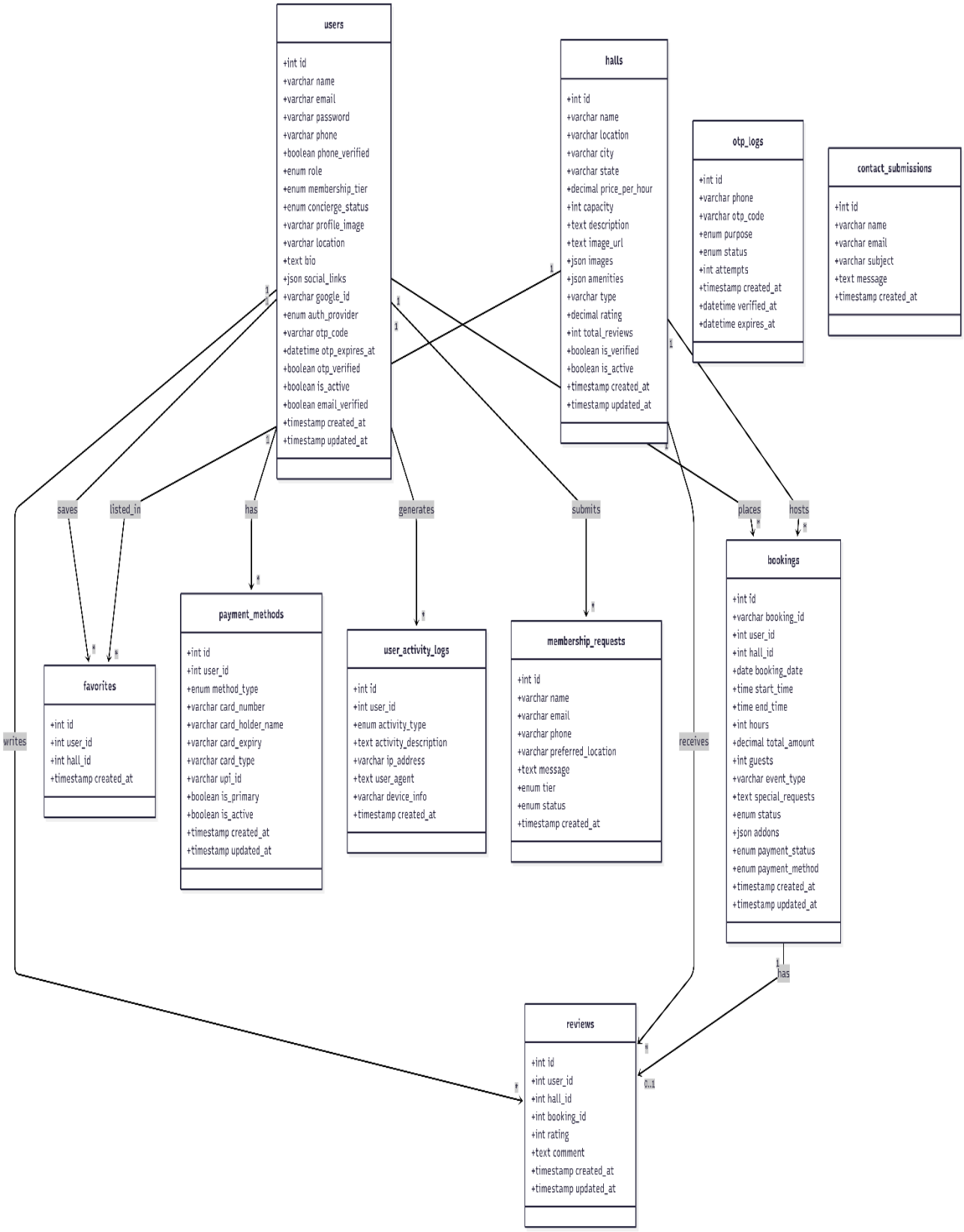


Figure 3.1.7.1 Class diagram of hall booking system

3.2 Database Design

3.2.1 Database Schema

Table 1: Users Table

Column	Data Type	Null	Keys & Constraints	Description
id	int	NN	PK (Auto Increment)	Unique user identifier
name	varchar (255)	NN		Full name of the user
email	varchar (255)	AN	Unique	Login email address
password	varchar (255)	AN		Hashed security password
phone	varchar (20)	AN	Unique	Verified phone number
phone_verified	boolean	NN	Default: false	Phone verification status
role	enum	NN	Default: 'user'	'user' or 'admin'
membership_tier	enum	NN	Default: 'classic'	'classic', 'silver', 'gold', 'platinum'
concierge_status	enum	NN	Default: 'none'	Priority service status
profile_image	varchar (500)	AN		URL/Path to profile picture
location	varchar (255)	AN		Home city or area
bio	text	AN		User's short biography
social_links	JSON	AN		Array of social media links
google_id	varchar (255)	AN	Unique	Google OAuth identity key
auth_provider	enum	NN	Default: 'local'	'local', 'google', or 'phone'
otp_code	varchar (6)	AN		Current 6-digit OTP code
otp_expires_at	datetime	AN		OTP validity expiration time
otp_verified	boolean	NN	Default: false	OTP check status
is_active	boolean	NN	Default: true	Account active/blocked status
email_verified	boolean	NN	Default: false	Email confirmation status
created_at	timestamp	NN		Registration timestamp
updated_at	timestamp	NN		Last profile update time

Table 3.9.4.1Table: Users

Table 2: halls (Venue Information) Table

Column	Data Type	Null	Keys & Constraints	Description
id	int	NN	PK (Auto Increment)	Hall unique ID
name	varchar (255)	NN		Name of the hall/banquet
location	varchar (255)	NN		Detailed map address
city	varchar (100)	NN		City name
state	varchar (100)	AN		State name
price_per_hour	decimal (10,2)	NN		Rental cost per hour
capacity	int	NN		Max guest limit
description	text	AN		Detailed venue description
image_url	text	AN		Main cover image URL
images	JSON	AN		Gallery of hall images
amenities	JSON	AN		List of features (WiFi, AC, etc.)
type	varchar (100)	NN	Default: 'Hall'	Palace, Penthouse, Ballroom, etc.
rating	decimal (3,2)	NN	Default: 4.5	Average customer rating
total_reviews	int	NN	Default: 0	Number of reviews received
is_verified	boolean	NN	Default: true	Verification badge status
is_active	boolean	NN	Default: true	Booking availability status
created_at	timestamp	NN		Time added to system
updated_at	timestamp	NN		Last venue update time

Table 3.9.4.2 Table: halls

Table 3: bookings (Reservations) Table

Column	Data Type	Null	Keys & Constraints	Description
id	int	NN	PK (Auto Increment)	Booking unique ID
booking_id	varchar (50)	NN	Unique	Tracking ID (e.g., BK-101)
user_id	int	NN	FK (users.id)	ID of the person booking
hall_id	int	NN	FK (halls.id)	ID of the reserved hall
booking_date	date	NN		Date of the event
start_time	time	NN		Event start hour
end_time	time	NN		Event finish hour
hours	int	NN		Total booking duration
total_amount	decimal (10,2)	NN		Final calculated price
guests	int	AN		Number of expected guests
event_type	varchar (100)	AN		e.g., Wedding, Corporate
status	enum	NN	Default: 'pending'	pending, confirmed, cancelled
addons	JSON	AN		Extra services (Catering, DJ)
payment_status	enum	NN	Default: 'pending'	pending, paid, refunded
payment_method	enum	NN	Default: 'Card'	Card, UPI, or Cash
created_at	timestamp	NN		Time of booking request

Table 3.9.4.3 Table: bookigs

Table 4: reviews (User Feedback) Table

Column	Data Type	Null	Keys & Constraints	Default Value & Description
id	int	NN	PK (Auto Increment)	Primary Key
booking_id	varchar (50)	NN	Unique	Professional Booking ID
user_id	int	NN	FK (Users.id)	Foreign Key
hall_id	int	NN	FK (Halls.id)	Foreign Key
booking_date	date	NN		Event Date
start_time	time	NN		Start Time
end_time	time	NN		End Time
hours	int	NN		Duration in hours
total_amount	decimal (10,2)	NN		Calculated revenue
guests	int	AN		Number of guests attending
status	enum	NN		'confirmed' (Default)
payment_status	enum	NN		'pending' (Default)

Table 3.9.4.4 Table: reviews

Table 5: favourite (Wishlist) Table

Column	Data Type	Null	Keys & Constraints	Description
id	int	NN	PK (Auto Increment)	Favourite unique ID
user_id	int	NN	FK (users.id)	User who liked the hall
hall_id	int	NN	FK (halls.id)	Saved hall ID
created_at	timestamp	NN		Time added to Wishlist

Table 3.9.4.5 Table: favorites

Table 6: otp_logs (Security Log) Table

Column	Data Type	Null	Keys & Constraints	Default Value & Description
id	int	NN	PK (Auto Increment)	Log primary ID
phone	varchar (20)	NN		Targeted mobile number
otp_code	varchar (6)	NN		The secret security code
purpose	enum	NN	Default: 'login'	registration, login, verification
status	enum	NN	Default: 'sent'	sent, verified, expired, failed
attempts	int	NN	Default: 0	Number of retry attempts
verified_at	datetime	AN		Time of successful verification
expires_at	datetime	AN		Code validity end time

Table 3.9.4.6 Table: otp_logs

Table 7: payment_methods (Saved Cards/UPI) Table

Column	Data Type	Null	Keys & Constraints	Default Value & Description
id	int	NN	PK (Auto Increment)	Payment record ID
user_id	int	NN	FK (users.id)	Owner's ID
method_type	enum	NN		'Card' or 'UPI'
card_number	varchar (20)	AN		Masked card digits
card_holder	varchar (255)	AN		Name on the card
card_expiry	varchar(7)	AN		Card expiry (MM/YYYY)
upi_id	varchar (255)	AN		User UPI handle
is_primary	boolean	NN	Default: false	Default payment choice

Table 3.9.4.7 Table: payment_methods

Table 8: user_activity_logs (Audit Trail) Table

Column	Data Type	Null	Keys & Constraints	Default Value & Description
id	int	NN	PK (Auto Increment)	Log unique ID
user_id	int	NN	FK (users.id)	ID of the acting user
activity_type	enum	NN		login, booking_created, etc.
description	text	AN		Detailed text of the action
ip_address	varchar (45)	AN		IP recorded for security
user_agent	text	AN		Device/Browser information
created_at	timestamp	NN		Event timestamp

Table 3.9.4.8 Table: user_activity_logs

Table 9: membership requests (Loyalty)Table

Column	Data Type	Null	Keys & Constraints	Default Value & Description
id	int	NN	PK (Auto Increment)	Request unique ID
name	varchar (255)	NN		Applicant's name
email	varchar(255)	NN		Applicant's email
phone	varchar (20)	AN		Contact phone
tier	enum	NN		'silver', 'gold', or 'platinum'
status	enum	NN	Default: 'pending'	pending, approved, rejected
message	text	AN		Extra notes from applicant
created_at	timestamp	NN		Request submission time

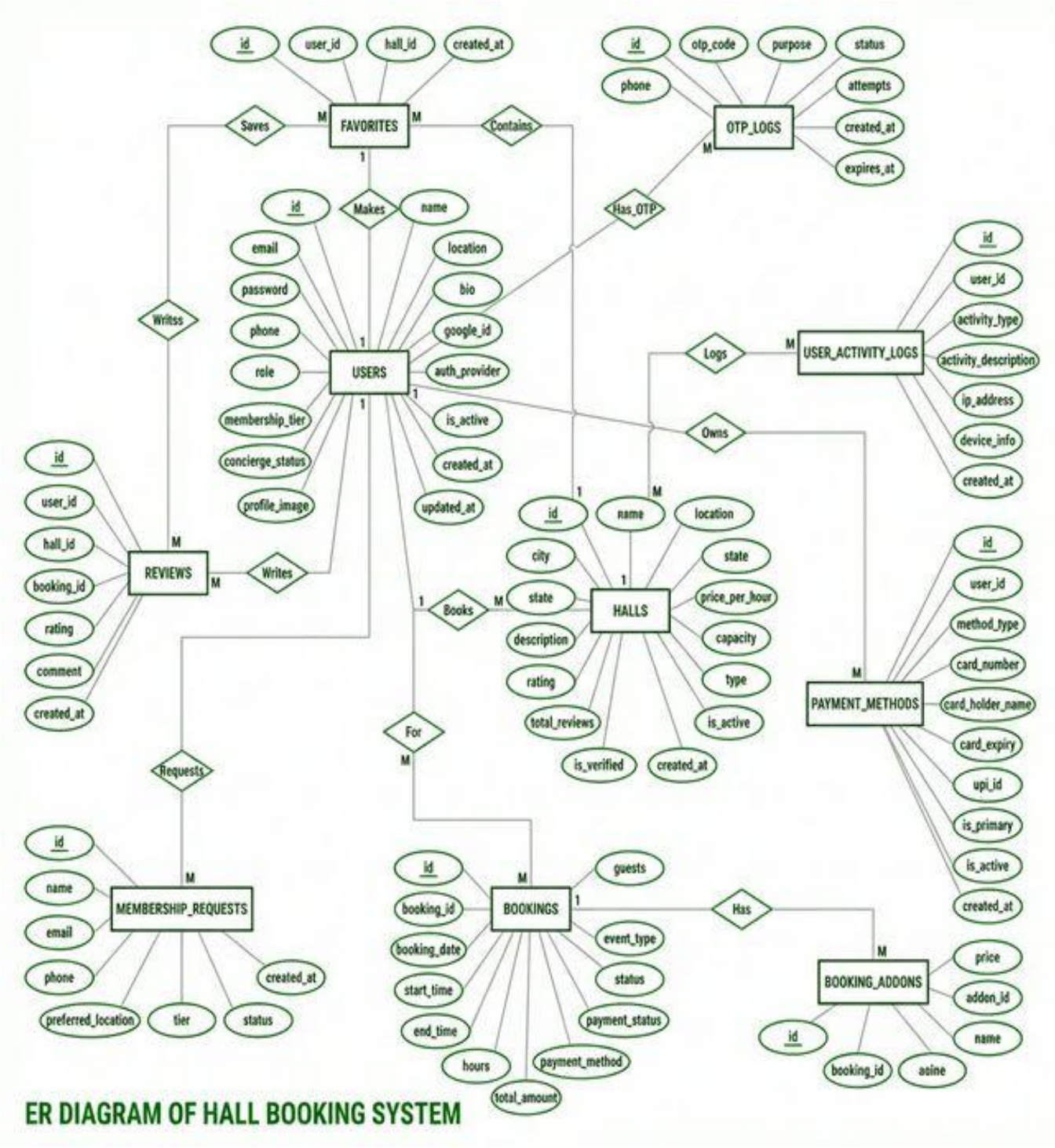
Table 3.9.4.9 Table: membership requests

Table 10: contact_submissions (Customer Queries) Table

Column	Data Type	Null	Keys & Constraints	Default Value & Description
id	int	NN	PK (Auto Increment)	Query unique ID
name	varchar(255)	NN		Sender's full name
email	varchar (255)	NN		Sender's email address
subject	varchar (255)	AN		Topic of the message
message	text	NN		Full inquiry text
created_at	timestamp	NN		Time inquiry was received

Table 3.9.4.10 Table: contact_submissions

3.2.2 E-R Diagram



CHAPTER 4

IMPLEMENTATION

4.1 Module Design and Specification

4.1.1 Screen-1: Registration Form

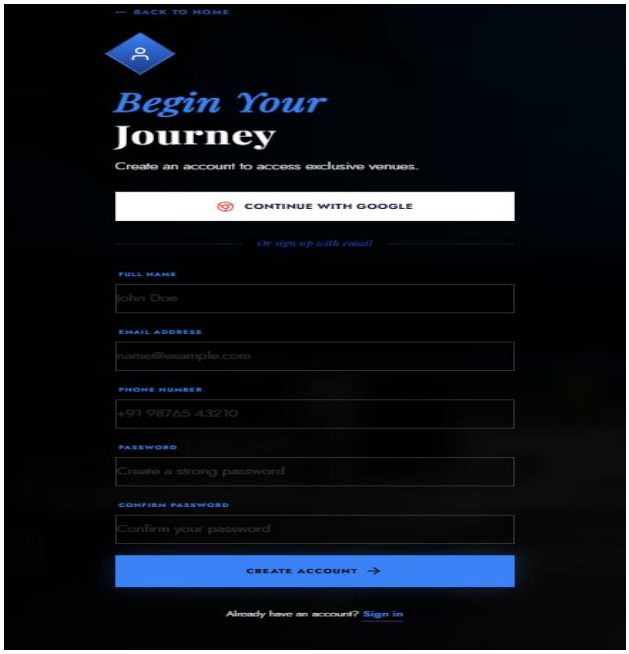


Figure 4.1 1 Screen-1: Registration Form

Purpose: This form will allow the target end-users to register in the system. To register, the following information will be encoded in the system.

Sr.	Screen Element	Input Type	O/M	Description
1	**Back to Home**	Link	------	Navigation link to return the user to the landing/home page.
2	**Continue with Google**	Button	------	Allows the user to quickly register using their Google OAuth account.
3	**Full Name**	Textbox	M	Field to enter the user's legal name; should be editable and accept text.
4	**Email Address**	Textbox	M	Field to enter a unique email address with proper @domain.com validation.
5	**Phone Number**	Textbox	M	Field to enter a contact number for OTP verification and profile security.
6	**Password**	Password	M	Field to create a strong password; masks input characters as dots for privacy.
7	**Confirm Password**	Password	M	Field used to re-enter the password to ensure both entries match exactly.

Table 4.1 1 Screen element of Registration form

Screen-2: Login Form

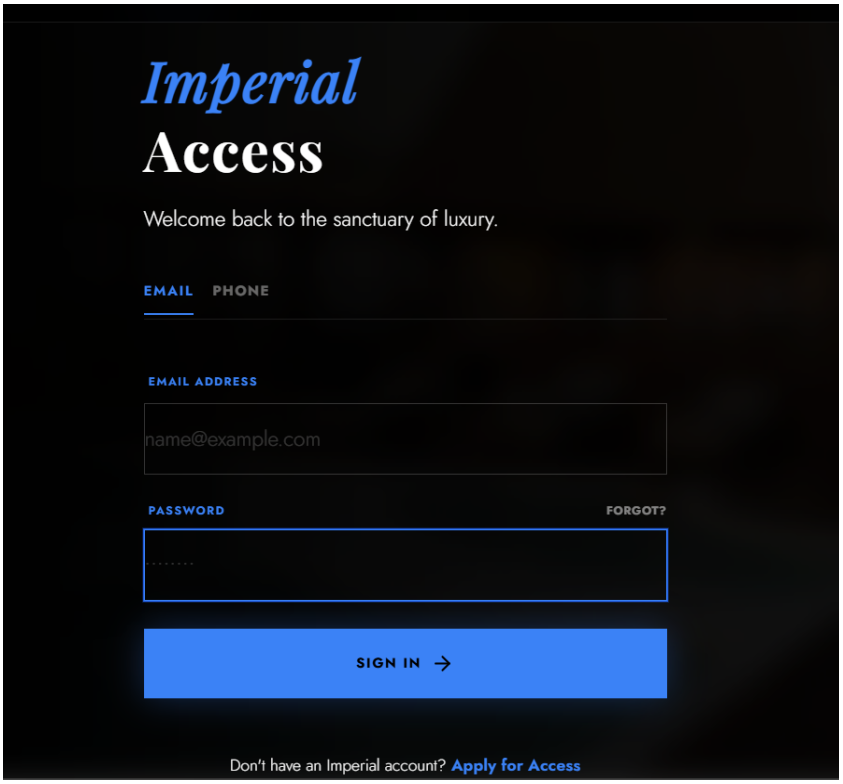


Figure4.1.2 Screen-2: Login Form

Purpose: This form will be used by the system's users to access records and features of the system. The users will input the correct combination of their username and password to be able to login to the system

Sr.	Screen Element	Input Type	O/M	Description
1	**Email Address**	Textbox	M	Email field should be editable and accept the user's registered email.
2	**Password**	Password	M	Password field should be editable and accept the password and display as dots.
3	**Forgot?**	Link	------	Clickable link for navigating to the Password Recovery page to recover a forgotten password.
4	**Apply for Access**	Link	------	Clickable link for navigating to the new member registration page.
5	**Sign In**	Button	------	Primary action button that validates login credentials and navigates to the dashboard.
6	**Email/Phone Tabs**	Tab	M	Interactive tabs to switch between Login via Email or Login via Phone Number.

Table 4.1.2 Screen element of Login form

4.1.2 Screen-3: Add Hall

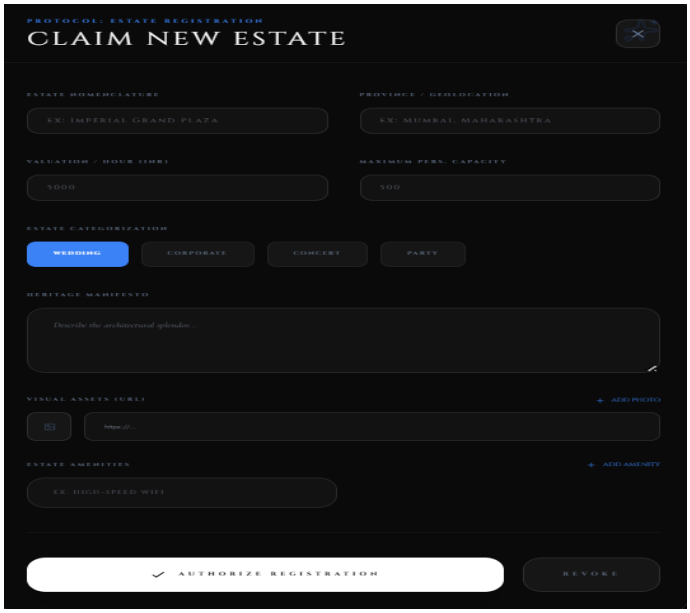


Figure 4.1.3 Screen-3: Add hall book

Purpose: This module will allow the system administrator to add, edit, update or delete of hall. The admin can add borrower information and manage it.

Sr.	Screen Element	Input Type	O/M	Description
1	**Estate Nomenclature**	**Textbox**	**M**	Field to enter the official name of the hall (e.g., Imperial Grand Palace).
2	**Province / Geolocation**	**Textbox**	**M**	Field to enter the city and state location of the venue.
3	**Valuation / Hour**	**Textbox**	**M**	Accepts numeric values for the hourly rental price in INR.
4	**Maximum Capacity**	**Textbox**	**M**	Accepts numeric values for the maximum guest limit of the hall.
5	**Estate Categorization**	**Tabs**	**M**	Selection buttons to categorize the hall (Wedding, Corporate, Concert, Party).
6	**Heritage Manifesto**	**Text area**	**M**	Multi-line textbox for writing a detailed description of the venue.
7	**Visual Assets (URL)**	**Textbox**	**M**	Field to enter the primary image URL link for the hall preview.
8	**Estate Amenities**	**Textbox**	**O**	Field to Enter specific features like "High-Speed WiFi" or "AC".
9	**Add Photo/Amenity**	**Link**	**------**	Clickable links to add multiple image URLs or amenity items.
10	**Authorize Registration**	**Button**	**------**	The primary "Submit" button to save the new hall data to the database.
11	**Revoke**	**Button**	**------**	The "Cancel" button to clear the form and stop the registration process.
12	**Close (X)**	**Button**	**------**	Icon button at the top right to close the registration dialog.

Table 4.1.3 Screen element of Add hall

4.2 Testing

4.2.1 Functional Testing

Test Case ID	Test Scenario	Test Steps	Expected Result	Status
TC001	User Login	1. Open Login Page 2. Enter valid email & password 3. Click "Sign In"	User is successfully logged in and redirected to Dashboard	Pass
TC002	Invalid Login	1. Enter incorrect email or password 2. Click "Sign In"	Error message displayed: "Invalid Credentials"	Pass
TC003	Add New Hall	1. Admin logs in 2. Navigates to "Claim New Estate" 3. Enters hall details & images 4. Clicks "Authorize"	Hall is successfully added to the public listings	Pass
TC004	Search for a Hall	1. User enters city name in search bar 2. Clicks "Search"	Matching halls in that city are displayed	Pass
TC005	Book a Hall	1. User selects an available slot 2. Fills event details 3. Clicks "Book Now"	Booking request is created with status "Pending"	Pass
TC006	Complete Payment	1. User goes to "My Bookings" 2. Selects a confirmed booking 3. Completes UPI/Card payment	Payment status is updated to "Paid" and receipt is generated	Pass
TC007	Cancel Booking	1. User visits the booking details 2. Clicks "Cancel Booking"	Booking status is updated to "Cancelled" and slot is freed	Pass
TC008	Admin Approval	1. Admin logs into panel 2. Views pending requests 3. Clicks "Approve"	Booking status changes to "Confirmed" for the user	Pass
TC009	Logout	1. Click on "Logout" button	User session is cleared and redirected to Login page	Pass

4.2.2 Non-Functional Testing

Test Case ID	Test Scenario	Test Steps	Expected Result	Status
TC-NF01	Performance (Load)	1. Simulate 100+ concurrent users searching for halls. 2. Perform simultaneous booking requests.	System handles the load without crashing; response time stays under 2 seconds.	Pass
TC-NF02	Security (Injection)	1. Enter SQL query strings in the email/login fields. 2. Try to bypass auth without a password.	System sanitizes inputs; access is denied and SQL Injection is prevented.	Pass
TC-NF03	Usability (Navigation)	1. Test UI with a new user. 2. Check if the "Book Now" buttons are easy to find.	User can complete a booking process within 5 clicks; UI is intuitive and clear.	Pass
TC-NF04	Usability (Responsive)	1. Open the website on iPhone, Android, and Tablets. 2. Check if the layout adjusts correctly.	All UI elements, images, and buttons scale perfectly on mobile screens.	Pass
TC-NF05	Reliability (Concurrency)	1. Two users attempt to book the exact same hall and slot at the same second.	System locks the slot for the first user; second user gets a "Slot already booked" alert.	Pass
TC-NF06	Security (MFA/OTP)	1. Attempt to login via Phone Number. 2. Enter an incorrect or expired OTP code.	System denies access and shows "Invalid or Expired OTP" message.	Pass

CHAPTER 5

CONCLUSION AND FUTURE ENHANCEMENT

5.1 Conclusion

The **Elite Hall Booking System** developed in this project successfully digitizes traditional manual venue management operations, making hall searching, reservations, and management more efficient. The system provides a seamless experience for both Administrators and Members, allowing them to manage and access venue resources with ease. Features like user authentication (via Google and OTP), real-time hall search, automated booking tracking, and membership loyalty tiers improve the efficiency and transparency of the entire process. The implementation of secure digital payment methods and role-based access control ensures that the system is professional, secure, and minimizes manual errors in scheduling.

Through this project, we have demonstrated how technology can simplify resource management in the hospitality and event sectors. By replacing manual paperwork and phone-based bookings with an automated, user-friendly system, this project significantly enhances business operations while reducing the administrative workload. The project aligns with modern digital solutions, making hall booking more accessible, streamlined, and scalable for future development.

5.2 Future Enhancement

While the current system covers the fundamental aspects of hall booking and venue management, several enhancements can be made to improve its functionality and user experience in the future:

1. **AI-Powered Venue Recommendations** Implementing Machine Learning algorithms to analyze user preferences and booking history to suggest the most suitable halls based on the event type, budget, and guest count.

2. **Virtual Reality (VR) 360° Tours** Integrating 360-degree virtual tours of halls to allow users to explore the venue remotely, providing a realistic view of the space and interior layout before making a reservation.

3. **Real-Time Live Chat & Concierge Bots** Enhancing user support by adding an AI-driven chatbot or live chat feature for instant query resolution, price negotiation, and priority assistance for premium membership tiers.

4. **Multi-Language and Global Currency Support** Expanding the platform to support multiple local and international languages, along with dynamic currency conversion, to cater to destination weddings and global events.

5. **Automated Catering and Vendor Integration** Adding modules for users to book catering, decoration, and photography services directly through the platform, creating an all-in-one event management solution.

APPENDICES

Appendix A: Acronyms and Abbreviations

Acronym	Full Form
HBS	Hall Booking System
DBMS	Database Management System (MySQL used in this project)
UI	User Interface (The modern frontend experience)
SQL	Structured Query Language (For database queries)
API	Application Programming Interface (To connect Frontend and Backend)
JWT	JSON Web Token (Used for secure user authentication)

REFERENCES

[1] **A. Srivastava & P. Kumar**, *Software Requirement Specification (SRS) for Hall Booking Management System*, SlideShare Inc., San Francisco USA, 2019, www.slideshare.net/AnirudhSrivastava/hall-booking-management-system-srs

[2] **Karl E. Wiegers**, *IEEE Standard 830-1998: SRS Template for Venue and Hall Reservation Systems*, IEEE Computer Society, web.cs.dal.ca/~hawkey/3130/srs_template-ieee.doc

[3] **Roger S. Pressman**, *Software Engineering: A Practitioner's Approach (Methodology for Hall Management Systems)*, McGraw-Hill, IA New York, 2010. www.processimpact.com/process_assets/srs_template.doc

[4] **P. J. Smith**, *Development of Online Multi-Purpose Hall Booking Systems*, ResearchGate, New Delhi, 2021. www.researchgate.net/publication/hall_booking_system_specification

[5] A. Silberschatz, H. F. Korth, and S. Sudarshan, Database System Concepts, 7th Edition, McGraw-Hill Education, New York, 2019.

[6] C. J. Date, An Introduction to Database Systems, 8th Edition, Pearson Education, Boston, MA, 2003.

[7] M. Jones, J. Bradley, and N. Sakimura, *RFC 7519 – JSON Web Token (JWT)*, Internet Engineering Task Force (IETF), 2015. https://datatools.ietf.org/html/rfc7519

[8] OWASP Foundation, *OWASP Top Ten Web Application Security Risks*, The Open Web Application Security Project, 2021. https://owasp.org/www-project-top-ten/

[9] L. Richardson and S. Ruby, *RESTful Web Services*, O'Reilly Media, Sebastopol, CA, 2007.

[10] R. T. Fielding, "Architectural Styles and the Design of Network-Based Software Architectures," PhD Dissertation, University of California, Irvine, 2000. https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm

[11] Meta Platforms Inc., *React.js Official Documentation – Building User Interfaces*, Meta Open Source, 2024. https://react.dev/learn

[12] OpenJS Foundation, *Node.js Documentation – Server-Side JavaScript Runtime*, OpenJS Foundation, 2024. https://nodejs.org/en/docs

[13] OpenJS Foundation, *Express.js – Fast, Unopinionated, Minimalist Web Framework for Node.js*, 2024. https://expressjs.com/